

Auditoría y prevención de fuga de datos en pipelines de Machine Learning generados por LLM

Witman Zavala Muñoz

Universidad de Las Américas

Facultad de Ingeniería y Negocios
Magíster en Ciencia de Datos

Capstone Project
Profesor Guía: Francisco Pérez
Profesor Co-guía: No aplica

Santiago de Chile, Octubre, 2026

RESUMEN

Los asistentes de programación basados en modelos de lenguaje grande (LLM) ya escriben pipelines de Machine Learning completos. Entre sus posibles errores, la fuga de datos es el único que no interrumpe la ejecución e infla las métricas de evaluación: un transformador (escalador, imputador, selector de variables) que se ajusta con todas las filas antes de separar entrenamiento y prueba produce un modelo que parece mejor de lo que es. Este informe presenta un auditor determinista que combina el análisis del árbol sintáctico (AST) con la verificación en ejecución, un refactorizador que repara los patrones habituales sin cambiar el modelo y un indicador de Sesgo de Optimismo ($\Delta\mathrm{Opt}$) que mide cuánto infla la fuga una métrica. Se evaluó sobre 96 pares de scripts con y sin fuga (6 datasets de scikit-learn, 4 familias de algoritmos y 4 tipos de fuga) y sobre 13 scripts escritos a mano como prueba independiente. El auditor acierta el 100 % del banco de control, que es una evaluación circular; en la prueba independiente el AST detecta 8 de 9 casos con fuga, la ejecución 7 de 9 y la combinación 9 de 9. El optimismo medido es casi nulo en clasificación (efecto techo) y de hasta 0,07 de R^2 en regresión; con señal nula la selección con fuga infla el AUC hasta 0,20. Los productos son dos Agent Skills y una plataforma web. Aún no se han generado pipelines con LLM reales; esa medición es el objetivo de Capstone II.

Para un público no técnico. Cuando una IA escribe un programa que aprende de datos, puede cometer un error silencioso: «espiar» las respuestas de la evaluación final mientras aprende, de modo que el resultado parece mejor de lo real. Este proyecto construye un revisor automático que encuentra ese error, lo corrige y calcula cuánto exageraba el resultado.

Palabras clave: fuga de datos; LLM; auditoría de código; scikit-learn; análisis estático; sesgo de optimismo.

ABSTRACT

LLM-based coding assistants already write end-to-end Machine Learning pipelines. Among their possible errors, data leakage is the only one that neither interrupts execution nor leaves evidence in the code, and it inflates evaluation metrics. This report presents a deterministic auditor that combines abstract syntax tree (AST) analysis with runtime verification, a refactoring tool that repairs common patterns without changing the model, and an Optimism Bias indicator ($\Delta\mathrm{Opt}$) that measures how much leakage inflates a metric. It was evaluated on 96 pairs of leaky and clean scripts (6 scikit-learn datasets, 4 algorithm families, 4 leakage types) and on 13 hand-written scripts as an independent test. The auditor scores 100 % on the control bank, which is a circular evaluation; on the independent test the AST detects 8 of 9 leaky cases, runtime verification 7 of 9, and their combination 9 of 9. Measured optimism is almost null in classification (ceiling effect) and up to 0.07 of R^2 in regression; with null signal, leaky feature selection inflates AUC by up to 0.20. The products are two Agent Skills and a web platform. No pipelines from real LLMs have been generated yet; that measurement is the goal of Capstone II.

Keywords: data leakage; LLM; code auditing; scikit-learn; static analysis; optimism bias.

Índice general

Resumen I
Abstract II

1. INTRODUCCIÓN Y FORMULACIÓN DEL PROBLEMA 1
1.1. Contexto 1
1.2. Qué es la fuga y cómo ocurre en scikit-learn 1
1.3. Problema, pregunta y relevancia 2
1.4. Objetivos 3
1.5. Métrica central y criterios de éxito 3
1.6. Alcance, supuestos y restricciones 3
1.7. Retroalimentación de la primera entrega: matriz de trazabilidad 4

2. FUNDAMENTACIÓN Y POSICIONAMIENTO 6
2.1. La fuga como sesgo de evaluación 6
2.2. Alternativas para detectar la fuga 6
2.3. Alternativas para medir el efecto y para la verdad de referencia 6
2.4. Brecha y decisiones de diseño 7

3. METODOLOGÍA, DATOS Y HOJA DE RUTA 8
3.1. Metodología: protocolo experimental en cinco fases 8
3.2. Datos: origen, obtención y caracterización 8
3.3. Diseño experimental y arquitectura por fase 9
3.4. Hoja de ruta actualizada y desviaciones 10

4. VALIDACIÓN, LÍNEA BASE Y RESULTADOS PRELIMINARES 11
4.1. Protocolo para evitar la fuga en la propia evaluación 11
4.2. Métricas y líneas base 11
4.3. Resultados: eficacia del auditor 11
4.4. Resultados preliminares: Sesgo de Optimismo 12
4.5. Control positivo: cuándo la selección con fuga sí daña 14

5. INTERPRETACIÓN Y JUICIO CRÍTICO 15

6. APLICABILIDAD Y USO RESPONSABLE 17
6.1. Contexto de uso 17
6.2. Propuesta de implementación 17
6.3. Riesgos éticos y medidas 17

7. CONCLUSIONES 19

REFERENCIAS 20

ANEXO A. REPRODUCIBILIDAD Y REPOSITORIO 23

1. INTRODUCCIÓN Y FORMULACIÓN DEL PROBLEMA

1.1 Contexto

Los asistentes de programación basados en modelos de lenguaje grande (LLM) ya escriben pipelines completos de ciencia de datos (Hong et al., 2025) y se evalúan con *benchmarks* de generación de código (Lai et al., 2023), y los *benchmarks* de agentes miden su puntaje o su tasa de éxito (Chan et al., 2025; Huang et al., 2024), no la validez metodológica de lo que producen. Un código generado por un LLM puede fallar de siete formas habituales: (1) errores de sintaxis; (2) alucinación de APIs o uso de métodos obsoletos; (3) incompatibilidad de dimensiones o tipos; (4) ineficiencia o agotamiento de memoria; (5) falta de control de semillas; (6) manejo inseguro de archivos o credenciales; y (7) **fuga de datos** (*data leakage*).

La fuga es el único de los siete que **no interrumpe la ejecución**. Los fallos 1 a 4 y 6 producen una excepción o son visibles para un *linter*, y el fallo 5 se detecta repitiendo la corrida. En cambio, la fuga produce un script que corre de principio a fin, entrega métricas más altas y, por eso, rara vez se cuestiona (Tabla 1.1). En la práctica humana el problema es amplio: Kapoor y Narayanan (2023) hallaron 294 artículos afectados por fuga en 17 campos científicos, y Kaufman et al. (2012) la formularon como información del objetivo que no estaría disponible al predecir.

Tabla 1.1. *Fallos ruidosos frente a fallo silencioso en código de ML generado por LLM.*

Dimensión	Fallos ruidosos (1–4 y 6)	Fuga de datos (7)
Comportamiento del intérprete	Lanza una excepción	Se ejecuta sin error ni advertencia
Detección estándar	Intérprete o *linter*	Invisible para herramientas convencionales: tipos y dimensiones coinciden
Efecto sobre las métricas	No llega a producirlas	Las infla (optimismo)
Riesgo en producción	Bajo: el pipeline no corre	Alto: el modelo se despliega con una capacidad de generalización peor que la creída

1.2 Qué es la fuga y cómo ocurre en scikit-learn

Un transformador (escalador, imputador, selector de variables, PCA) *aprende* parámetros con `.fit()`: medias y desviaciones, vecinos más cercanos, o qué variables conservar. Si aprende con **todas** las filas antes de separar entrenamiento y prueba, o fuera de un `Pipeline` dentro de una validación cruzada, ya vio las filas que luego se usan para evaluar. Formalmente, el aislamiento correcto exige que los parámetros del preprocesamiento sean función solo de los datos de entrenamiento; hay fuga cuando algún parámetro se estima con la unión de entrenamiento y prueba. La Figura 1.1 contrasta ambas formas con el mismo modelo, los mismos datos y los mismos pliegues.

Para ver el mecanismo con números, el script `leakage_demo.py` ejecuta ambas formas con `breast_cancer`, un `KNNImputer`, un escalador, `SelectKBest` y una red `MLPClassifier`, con los mismos pliegues (Figu-

```
# CON FUGA: el escalador ve todas las filas
X_esc = StandardScaler().fit_transform(X)
scores = cross_val_score(modelo, X_esc, y, cv=5)

# SIN FUGA: en cada pliegue, .fit() ve solo el entrenamiento
pipe = Pipeline([("escalador", StandardScaler()), ("modelo", modelo)])
scores = cross_val_score(pipe, X, y, cv=5)
```

Figura 1.1. *Misma operación con y sin fuga de datos.*

ra 1.2). Con datos reales, el ROC-AUC es 0,979 con fuga y 0,977 sin ella: $\Delta\mathrm{Opt} = +0{,}0015$, casi invisible porque la señal es muy fuerte. Con 500 columnas de ruido y etiquetas permutadas (verdad: AUC $= 0{,}5$), sube a 0,571 frente a 0,523 ($\Delta\mathrm{Opt} = +0{,}048$): el selector «encuentra» columnas que por azar parecen predecir.

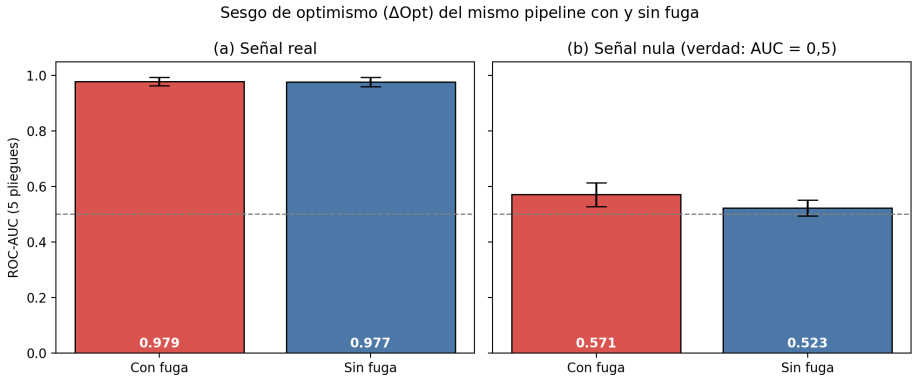


Figura 1.2. *Mismo pipeline con y sin fuga. (a) Señal real. (b) Señal nula; la línea punteada marca el AUC verdadero (0,5).*

Se estudian cuatro mecanismos: **escalado** (la media y la varianza incluyen filas de prueba), **imputación** (`KNNImputer` puede copiar valores de vecinos de prueba), **selección de variables** (`SelectKBest` usa las etiquetas de todas las filas) y **combinada** (los tres a la vez).

1.3 Problema, pregunta y relevancia

Problema. Un script de ML tabular puede contener fuga de datos sin que nada lo advierta, y quien recibe ese código de un asistente de IA no tiene una forma barata de comprobarlo ni de saber cuánto distorsiona la evaluación.

Pregunta de investigación. ¿Es posible detectar automáticamente, leyendo y ejecutando el código, la fuga de preprocesamiento y selección en pipelines de scikit-learn; reparar el código sin cambiar el modelo; y cuantificar cuánto optimismo introduce la fuga en distintas familias de algoritmos?

Relevancia. Los detectores existentes son principalmente estáticos y se evaluaron sobre código humano (AlOmar et al., 2025; Drobnjaković et al., 2024; Yang et al., 2022), y los LLM tienen dificultades para identificar por sí mismos estos problemas tempranos del pipeline (Kumar et al., 2025). Falta un instrumento determinista que audite código de asistentes, lo repare y mida el daño; ese es el entregable de este proyecto.

1.4 Objetivos

Objetivo general. Diseñar, validar empíricamente e implementar un marco automatizado de detección y prevención de fuga de datos en pipelines de aprendizaje supervisado con scikit-learn, y cuantificar el sesgo de optimismo que induce en cuatro familias de algoritmos.

Tabla 1.2. *Objetivos específicos verificables y evidencia.*

OE	Objetivo específico	Evidencia
OE1	Formalizar los mecanismos de fuga (escalado, imputación, selección, combinada) y distinguirlos de otros fallos de código generado.	Secciones 1.1–1.2; banco de 96 pares
OE2	Desarrollar un auditor determinista (AST y verificación en ejecución) y medir su precisión, recall y VPN contra una verdad conocida.	Tablas 4.1 y 4.2
OE3	Cuantificar el Sesgo de Optimismo $\Delta\mathrm{Opt}$ por tipo de fuga y familia (RandomForest, Gradient Boosting, lineal regularizado y red neuronal MLP) en 3 datasets de clasificación y 3 de regresión.	Tablas 4.3 y 4.4; Figuras 4.1 y 4.2
OE4	Entregar dos Agent Skills y una plataforma web que permitan auditar, reparar y crear pipelines sin fuga.	Capítulo 6

1.5 Métrica central y criterios de éxito

La métrica es el **Sesgo de Optimismo Inducido por Fuga**:

$$\Delta\mathrm{Opt} = M_{\mathrm{leaky}} - M_{\mathrm{clean}}, \tag{1.1}$$

donde M es el promedio de una métrica (ROC-AUC, F1, R^2 o $-$RMSE) en 5 pliegues, y M_{leaky} y M_{clean} provienen del mismo dataset, los mismos pliegues (semilla 42), el mismo modelo y los mismos hiperparámetros: solo cambia si el preprocesamiento se ajustó con todas las filas o dentro de un `Pipeline`. Por construcción, si no hay fuga, $\Delta\mathrm{Opt} = 0$. La sección 2.3 justifica por qué se descarta la «brecha de generalización», que mide sobreajuste y no fuga.

Donde: $\Delta\mathrm{Opt}$: sesgo de optimismo; M_{leaky}: métrica promedio del pipeline con fuga; M_{clean}: métrica promedio del mismo pipeline sin fuga (preprocesamiento dentro de un `Pipeline`); un valor positivo indica que la fuga mejora artificialmente la métrica.

Nota. H-esc.: hipótesis del escalado en árboles; H-MLP: hipótesis de mayor vulnerabilidad de la red neuronal; FP: falsos positivos. Los umbrales son propuestas del autor, formalizadas después de la prueba piloto descrita aquí; en Capstone II se fijarán antes de evaluar con pipelines de LLM reales.

1.6 Alcance, supuestos y restricciones

Alcance: clasificación y regresión supervisadas con scikit-learn sobre datos tabulares; fuga de preprocesamiento, imputación y selección de variables. El *clustering* queda fuera porque no hay un objetivo explícito contra el cual medir el optimismo. *Supuestos:* datos i.i.d.; 5 pliegues con semilla fija; el generador de pares

Tabla 1.3. *Criterios de éxito operativos y resultado (detalle en los capítulos 4 y 5).*

OE	Criterio (umbral o punto de comparación)	Resultado
OE2	Banco de control: precisión y recall $\geq 0{,}90$	1,00 y 1,00 (evaluación **circular**, ver sección 4.3)
OE2	Evaluación independiente (13 scripts manuales): recall híbrido $\geq 0{,}80$ y 0 falsos positivos	9/9 y 0 FP; IC95 % exacto del recall [0,66; 1,00]: **con** $n = 9$ **no se puede afirmar** el umbral
OE3	H-esc.: en árboles, $\Delta\mathrm{Opt}$ por escalado ≈ 0 ($\lvert\Delta\mathrm{Opt}\rvert < 0{,}001$)	Confirmada: máximo 0,0002
OE3	H-MLP: la red neuronal sufre el mayor $\Delta\mathrm{Opt}$	**Parcial**: solo en regresión y sin significancia tras corregir por multiplicidad
OE4	Reparación automática ≥ 90 % de los scripts con fuga	96/96 en el banco; **4/9** en la evaluación independiente

reproduce con fidelidad el mecanismo de fuga. *Restricciones:* solo datasets nativos de scikit-learn y sin anotadores humanos (observaciones del comité); no se auditan fugas temporales ni de dominio; los pipelines escritos por LLM reales **aún no se han generado** (Capstone II).

1.7 Retroalimentación de la primera entrega: matriz de trazabilidad

Las observaciones del comité apuntan a validar primero los cimientos (problema, métrica, datos y protocolo) y a eliminar la variabilidad no controlada, como los anotadores humanos. La Tabla 1.4 vincula cada una con su diagnóstico, la solución y la sección donde se verifica. En la forma, toda figura y tabla tiene *caption* numerado y se cita en el texto, y cada cifra proviene de los archivos de resultados del repositorio.

Tabla 1.4. *Matriz de trazabilidad entre las observaciones del comité y su resolución.*

Cód.	Requerimiento del comité	Diagnóstico	Solución adoptada	Dónde
R1	Contextualizar los fallos de los LLM y justificar por qué estudiar solo la fuga	La elección parecía arbitraria, sin una taxonomía de defectos	Distinción entre fallos ruidosos (*fail-stop*) y el fallo silencioso de la fuga, único que no detiene la ejecución e infla la métrica	Sec. 1.1, Tabla 1.1
R2	Especificar qué modelos generan y cuáles auditan	Riesgo de sesgo de autoconfirmación si un LLM se evalúa a sí mismo	Separación estricta: LLM generadores como sujetos de estudio; auditor determinista (AST y ejecución) que nunca consulta a un LLM	Sec. 3.3, Tabla 3.3
R3	Mostrar cómo se fugan los datos con un ejemplo simple	La formulación abstracta impedía ver el mecanismo	Ejemplo de código con y sin fuga y script comentado paso a paso que genera un gráfico comparativo	Sec. 1.2, Figuras 1.1 y 1.2
R4	Explicar el experimento en la práctica, paso a paso	El flujo anterior de 19 etapas era difícil de auditar	Protocolo lineal de cinco fases deterministas, cada una con un script y un artefacto verificable	Sec. 3.1, Tabla 3.1
R5	Usar datasets de scikit-learn y evitar anotadores humanos	Datos externos y etiquetado manual agregan variabilidad subjetiva	Seis datasets nativos de `sklearn.datasets`; verdad conocida por construcción (oráculo programático)	Sec. 3.2 y 3.1
R6	Elegir el paradigma y las familias de modelos	Faltaba acotar la tarea y justificar la sensibilidad de cada familia	Clasificación y regresión supervisadas; cuatro familias (árboles, *boosting*, lineal regularizado y MLP) con hipótesis explícitas	Sec. 1.5–1.6
R7	Dejar todo en GitHub y poder explicar el código	Se requería trazabilidad computacional y dominio técnico	Repositorio público modular, un comando de reproducción y scripts comentados por bloques	Anexo A
R8	Definir como producto una o varias *Skills*	Faltaba un entregable transferible	Dos Agent Skills y una plataforma web en Streamlit como alternativa gráfica	Cap. 6
R9	Validar la métrica («brecha de generalización auditada»)	El nombre se confundía con el sobreajuste	Sesgo de Optimismo Inducido por Fuga, $\Delta\mathrm{Opt}$, medido con pliegues idénticos	Sec. 1.5

2. FUNDAMENTACIÓN Y POSICIONAMIENTO

El proyecto exige tres decisiones que la literatura permite justificar: cómo *detectar* la fuga, cómo *medir* su efecto y contra qué *verdad de referencia* validar el detector. Este capítulo compara las alternativas para cada una y deriva de esa comparación la brecha y las decisiones de diseño.

2.1 La fuga como sesgo de evaluación

Kaufman et al. (2012) definen la fuga como el uso, durante el entrenamiento o la evaluación, de información que no estaría disponible al momento de predecir. En preprocesamiento, el caso más estudiado es la selección de variables: Ambroise y McLachlan (2002) mostraron que elegir genes con todas las muestras y validar después produce tasas de error engañosamente bajas, y Varma y Simon (2006) y Cawley y Talbot (2010) demostraron que optimizar decisiones con los mismos datos con que se estima el desempeño sesga esa estimación hacia el optimismo. Hastie et al. (2009) lo resumen como la forma «incorrecta» y la «correcta» de validar: todo paso que mira los datos debe repetirse dentro de cada pliegue. La amplitud del problema en la práctica es conocida: Kapoor y Narayanan (2023) documentaron 294 artículos afectados en 17 campos. Esta literatura sustenta dos hipótesis del proyecto: la selección supervisada debería ser la fuga más dañina cuando la señal es débil, y un transformador afín, como el escalado, no debería afectar a los árboles.

2.2 Alternativas para detectar la fuga

La Tabla 2.1 compara cuatro enfoques. Los **detectores estáticos** analizan el flujo de datos de notebooks (Yang et al., 2022), lo formalizan con interpretación abstracta (Drobnjaković et al., 2024) o lo integran a un IDE (AlOmar et al., 2025). Los **LLM como detector** combinan *slicing* con *prompts* estructurados (Draoui et al., 2026) o integran un agente de chequeo de fuga (Nam et al., 2025), pero como jueces prefieren sus propias respuestas (Zheng et al., 2023) y fallan al identificar problemas tempranos del pipeline (Kumar et al., 2025). Las **listas de verificación** (Kapoor et al., 2024; Lones, 2024) dependen de un revisor experto. La **instrumentación en ejecución** observa con cuántas filas se ajustó cada transformador, a costa de ejecutar el código.

2.3 Alternativas para medir el efecto y para la verdad de referencia

Para **medir el daño**, la *brecha de generalización* (entrenamiento menos prueba) mide sobreajuste, que existe aun sin fuga (por ejemplo, en un árbol sin poda), y confunde dos fenómenos. La *validación cruzada anidada* corrige el sesgo de selección de modelos (Cawley & Talbot, 2010; Varma & Simon, 2006), pero no aísla el aporte de un error concreto del código. La *comparación pareada* de una evaluación defectuosa con su versión corregida, con los mismos pliegues, sí lo aísla, porque todo lo demás queda fijo; es la lógica de los reanálisis de Kapoor y Narayanan (2023) y la que adopta $\Delta\mathrm{Opt}$.

Tabla 2.1. *Síntesis comparativa de los enfoques para detectar fuga de datos.*

Enfoque	Fortaleza	Limitación	¿Mide el daño?	Uso en este proyecto
Análisis estático (flujo de datos)	Determinista, reproducible, no ejecuta código	Ciego a fugas que dependen de valores y a alias; validado en código humano	No	Núcleo del auditor (AST)
LLM como detector o juez	Entiende estilos variados de código	Sesgo de autopreferencia, variabilidad entre corridas, sin evidencia trazable	No	Descartado como instrumento
Listas de verificación	Cubren fugas temporales y de dominio	Requieren un revisor experto; no se automatizan	No	Criterio de «código correcto»
Instrumentación en ejecución	Observa el ajuste real, inmune a alias	Exige ejecutar código no confiable	No	Segunda opinión del auditor
Este proyecto (híbrido)	Lee y ejecuta; repara patrones comunes	Solo scikit-learn tabular; reparación limitada	**Sí** ($\Delta\mathrm{Opt}$)	—

Para la **verdad de referencia**, los anotadores humanos introducen variabilidad subjetiva y el LLM juez hereda los problemas de la Tabla 2.1. El oráculo programático, que construye cada script con y sin fuga, conoce la etiqueta con certeza, pero si el mismo autor escribe el generador y el detector la evaluación es circular; por eso se complementa con un conjunto escrito a mano con estilos que el generador no produce.

2.4 Brecha y decisiones de diseño

Entre las fuentes revisadas, ninguna mide el optimismo mediante una intervención pareada sobre el mismo pipeline ni combina la lectura y la ejecución del código con su reparación. La Tabla 2.2 resume las decisiones que se derivan de la comparación anterior.

Tabla 2.2. *Decisiones de diseño, alternativas descartadas y respaldo.*

Decisión	Alternativa descartada	Justificación	Respaldo
Auditor determinista (AST y ejecución)	LLM como juez	Un instrumento de medición debe dar el mismo veredicto en cada corrida y no puede tener interés en el resultado	Zheng et al. (2023)
Detector híbrido	Solo estático o solo dinámico	El AST no ve valores ni alias; la ejecución no ve ajustes parciales: fallan en casos distintos	Yang et al. (2022)
$\Delta\mathrm{Opt}$ pareado (mismos pliegues)	Brecha de generalización; CV anidada	Aísla causalmente el efecto del error; la brecha mezcla fuga y sobreajuste	Cawley & Talbot (2010) y Kapoor & Narayanan (2023)
Oráculo programático más un conjunto manual independiente	Anotadores humanos o LLM juez	Etiqueta exacta y sin subjetividad; el conjunto manual controla la circularidad	Hevner et al. (2004)
Columnas de ruido y control positivo con señal nula	Solo datos originales	La fuga por selección solo se manifiesta si hay variables inútiles entre las cuales elegir	Ambroise & McLachlan (2002)
Pruebas no paramétricas pareadas con corrección de Holm	t de Student sin corregir	Pocos datasets y sin supuesto de normalidad; varias pruebas simultáneas	Demšar (2006) y Holm (1979)

3. METODOLOGÍA, DATOS Y HOJA DE RUTA

3.1 Metodología: protocolo experimental en cinco fases

Se adapta CRISP-DM a la construcción de un instrumento de medición: siguiendo la lógica de *design science*, se construye un artefacto y se evalúa contra una verdad conocida (Hevner et al., 2004). El flujo anterior, de 19 etapas, se reemplazó por un protocolo lineal de cinco fases deterministas (Tabla 3.1). Cada fase tiene un propósito ligado a un objetivo específico, un script que la ejecuta y un artefacto que la deja verificable; todas se reproducen con `python run_all.py`.

Tabla 3.1. *Protocolo en cinco fases: propósito, implementación y evidencia.*

Fase	CRISP-DM	Propósito	Script	Artefacto	OE
0. Datos	Comprensión y preparación	Cargar y caracterizar 6 datasets; ruido y faltantes reproducibles	`sklearn_loader.py`; `eda_datasets.py`	`eda_datasets.csv`	OE3
1. Banco de control	Preparación	Escribir 96 pares con y sin fuga (oráculo programático)	`generate_controlled_pairs.py`	192 scripts	OE1
2. Auditoría	Modelado del instrumento	Auditar cada script con AST y con verificación en ejecución	`03_auditor/`; `run_benchmark.py`	`auditor_per_script.csv`	OE2
3. Eficacia del auditor	Evaluación	Comparar veredictos con la verdad conocida y con la línea base	`run_benchmark.py`; `independent_evaluation.py`	Tablas 4.1–4.2	OE2
4. Sesgo de optimismo	Evaluación	Calcular $\Delta\mathrm{Opt}$ por par, pruebas por familia y control positivo	`statistical_analysis.py`; `noise_sensitivity.py`	Tablas 4.3–4.4	OE3
5. Empaquetado	Despliegue	Publicar el auditor como dos Skills y una plataforma web	`05_skills/`; `06_web_app/`	Cap. 6	OE4

Donde: OE: objetivo específico. La fase 0 prepara los insumos de las cinco fases experimentales.

3.2 Datos: origen, obtención y caracterización

Se usan seis conjuntos nativos de scikit-learn (Tabla 3.2): tres de clasificación (`breast_cancer`, `wine`, `digits`) y tres de regresión (`diabetes`, `california_housing`, `make_friedman1`). Son públicos, no contienen datos personales y se cargan sin copiarlos al repositorio; solo `california_housing` se descarga la primera vez. La Figura 3.1 resume la dispersión de escalas y el balance de clases.

Tabla 3.2. *Caracterización de los datasets.*

ID	Dataset	Tarea	N	Variables	Razón de escalas	Asimetría media	Clase minoritaria
D1	breast_cancer	Clasif. (2)	569	30	215.171	1,74	37,3 %
D2	wine	Clasif. (3)	178	13	2.530	0,43	27,0 %
D3	digits	Clasif. (10)	1.797	64	277	4,89	9,7 %
D4	diabetes	Regresión	442	10	1,0	0,41	—
D5	california_housing	Regresión	3.000*	8	1.398	10,00	—
D6	friedman1	Regresión	1.000	50	1,1	0,04	—

Nota. Razón de escalas: desviación estándar máxima sobre mínima entre variables. *Submuestra determinista de 3.000 filas de

20.640 (desviación B1, sección 3.4). `digits` tiene 3 columnas constantes.

Calidad más allá de los faltantes. Ningún dataset base tiene filas duplicadas. La proporción de celdas atípicas ($|z| > 3$ por columna) es baja: 1,24 % en `breast_cancer`, 0,48 % en `wine`, 0,67 % en `digits`, 0,34 % en `diabetes`, 0,59 % en `california_housing` y 0 % en `friedman1` (`eda_datasets.csv`). Son datos limpios por diseño, y por eso los faltantes se simulan.

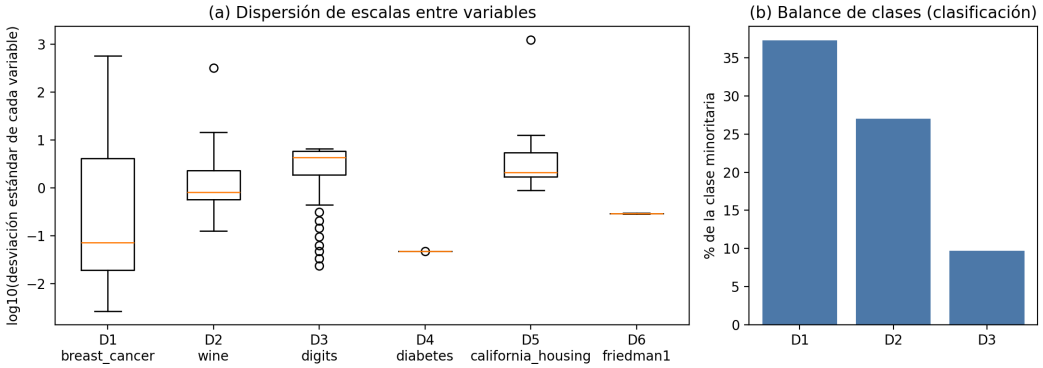


Figura 3.1. *(a) Dispersión de escalas entre variables, en $\log_{10}$ de la desviación estándar. (b) Porcentaje de la clase minoritaria en los datasets de clasificación.*

Preparación trazable. (i) *Variante «completo»*: sin faltantes. (ii) *Variante «con_faltantes»*: se borra el 10 % de las celdas con semilla fija (media observada entre 9,8 y 10,1 %). (iii) A D1–D5 se les agregan 50 columnas de ruido gaussiano, porque la fuga por selección solo se manifiesta si hay variables inútiles entre las cuales elegir; D6 ya trae 45. Todo usa la semilla 42, y dos ejecuciones producen datos idénticos (verificado en las pruebas automáticas).

Representatividad y limitaciones. Son datasets clásicos, limpios, pequeños y tabulares; no representan datos reales ruidosos, de gran escala ni con estructura temporal o jerárquica, y los LLM probablemente los vieron durante su entrenamiento. Por la dispersión de escalas (Figura 3.1a), el escalado debería afectar sobre todo a los modelos sensibles a la escala; por la asimetría de D5 (10,0) se esperan valores atípicos que desestabilicen a la red neuronal, y así ocurrió (sección 3.4). Con ROC-AUC cercana a 1 en D1–D3 hay un efecto techo que limita cuánto optimismo puede observarse en clasificación.

3.3 Diseño experimental y arquitectura por fase

Para cada combinación de 6 datasets $\times$ 4 familias $\times$ 4 tipos de fuga, un generador escribe un script con fuga (`leaky`) y otro idéntico sin ella (`clean`): **96 pares, 192 scripts**. Los modelos son `RandomForest`, `HistGradientBoosting`, `LogisticRegression` o `Ridge`, y `MLPClassifier` o `MLPRegressor`, todos de scikit-learn (Pedregosa et al., 2011). Como la verdad se conoce por construcción, no se necesitan anotadores.

Cada script se audita con **dos mecanismos independientes**. El primero analiza el árbol sintáctico (`ast`) con tres reglas: *R-Split* (ajuste antes de `train_test_split`), *R-CV* (ajuste fuera de un `Pipeline` cuyo resultado entra a la validación cruzada) y *R-Test* (ajuste con una variable que parece de prueba). El segundo es un verificador en ejecución que cuenta con cuántas filas se ajusta cada transformador y declara fuga si coincide con el total del dataset. El detector híbrido declara fuga si cualquiera de los dos la ve.

Para evitar el sesgo de autopreferencia, el protocolo separa a los sujetos de estudio del sistema de medición (Tabla 3.3): ningún LLM participa en la auditoría ni en el cálculo de las métricas.

Tabla 3.3. *Arquitectura por fase: quién genera y quién audita.*

Componente	Rol	Implementación	¿Usa LLM?
Conjunto A (generador)	Sujetos de estudio: pipelines escritos por LLM	Mismo *prompt* para todos; versión exacta y fecha registradas (Capstone II)	Sí
Conjunto B (banco de control)	Verdad conocida por construcción	`generate_controlled_pairs.py`: 96 pares con y sin fuga	No
Conjunto manual	Control de la circularidad	13 scripts escritos a mano con estilos que el generador no produce	No
Motor auditor	Instrumento de medición	AST (tres reglas) y verificación en ejecución; salida idéntica en cada corrida	No

3.4 Hoja de ruta actualizada y desviaciones

Tabla 3.4. *Hoja de ruta con hitos y entregables (estado a octubre de 2026).*

Periodo	Actividad	Hito o entregable verificable	Estado
Capstone I	Reformulación del problema y la métrica; datasets; banco de 96 pares; auditor AST y de ejecución; refactorizador; $\Delta\mathrm{Opt}$; Skills y plataforma web (prototipo)	Repositorio público reproducible con un comando; este informe	Hecho
Capstone II, sem. 1–4	Generar pipelines con 6–8 LLM reales (24 tareas por modelo) y auditarlos; ejecutarlos en un contenedor aislado	Conjunto A versionado; tasa de fuga por modelo con IC95 %	Pendiente
Capstone II, sem. 5–8	Mejorar el auditor (alias de importación, funciones auxiliares, `ColumnTransformer`); ampliar la evaluación independiente de 13 a ≥ 60 scripts	Auditor v2 con recall independiente $\geq 0{,}80$ (umbral fijado antes de evaluar)	Pendiente
Capstone II, sem. 9–12	Datasets con señal débil; $\Delta\mathrm{Opt}$ por tramo de señal/ruido; piloto de uso con 3–5 personas	Tabla de $\Delta\mathrm{Opt}$ condicionada; informe del piloto	Pendiente
Capstone II, sem. 13–16	Redacción final y defensa	Documento final y presentación	Pendiente

Desviaciones respecto del plan original (justificación completa en `07_report/decision_log.md`):

- **B1.** `california_housing` se submuestrea a 3.000 filas: `KNNImputer` crece con n^2 y el banco de 192 scripts no sería ejecutable en un portátil.
- **B2.** El sobremuestreo (SMOTE) no se mide, porque no pertenece a scikit-learn; el cuarto tipo es «combinada».
- **C1.** En la primera corrida, los modelos lineal y MLP sin escalar daban diferencias negativas por inestabilidad numérica (hasta $-0{,}155$ en `wine`); se añadió un escalador interno idéntico en ambas versiones (corrida descartada en `results/run1_discarded/`).
- **C2.** Un `MLPRegressor` divergió en un pliegue ($R^2 = -21$ en `california_housing`; promedio $-3{,}86$). Se definió, **después de ver el problema**, que un par es degenerado si algún pliegue tiene $R^2 < -1$; se excluyó 1 par de 96 y se lista aparte.

4. VALIDACIÓN, LÍNEA BASE Y RESULTADOS PRELIMINARES

4.1 Protocolo para evitar la fuga en la propia evaluación

Se aplica al experimento lo que se audita: (i) validación cruzada de 5 pliegues (estratificada en clasificación) con semilla 42 y los **mismos pliegues** para `leaky` y `clean`; (ii) en la versión `clean` y en el código reparado, todo el preprocesamiento va dentro de un `Pipeline`, de modo que se ajusta solo con el entrenamiento de cada pliegue; (iii) el verificador en ejecución comprueba que, en `clean`, ningún transformador vea tantas filas como el total (el máximo observado es 80,3 %); (iv) las pruebas estadísticas usan como unidad el **par** (dataset $\times$ tipo de fuga) y no los pliegues, que no son independientes.

4.2 Métricas y líneas base

Las métricas se derivan de los criterios de éxito (Tabla 1.3). *Del auditor* (positivo $=$ «hay fuga»): precisión, recall, F1 y valor predictivo negativo (VPN), porque el error más costoso es declarar limpio un script con fuga (falsa seguridad); el recall de la evaluación independiente se acompaña de un intervalo exacto de Clopper–Pearson (Clopper & Pearson, 1934), adecuado para muestras pequeñas. *Del daño*: $\Delta\mathrm{Opt}$ en ROC-AUC (clasificación, insensible al umbral y al desbalance) y R^2 (regresión, comparable entre datasets), con F1 macro y RMSE como secundarias; la media de $\Delta\mathrm{Opt}$ por familia se informa con un intervalo bootstrap percentil del 95 % (Efron & Tibshirani, 1993).

Se definen tres líneas base, cada una como punto de comparación de una afirmación distinta:

- **Del daño:** el pipeline `clean`. Es el desempeño que se habría reportado sin el error, por lo que $\Delta\mathrm{Opt}$ mide exactamente el exceso atribuible a la fuga.
- **Del detector híbrido:** cada componente por separado (solo AST y solo ejecución). Justifica que combinar ambos aporte algo.
- **Externa:** una regla trivial (`baseline_simple_rule.py`) que declara fuga si el script ajusta un transformador con `.fit` y no usa ningún `Pipeline`, ignorando comentarios e importaciones. Representa lo que haría un revisor apurado con una búsqueda de texto; si el auditor no la supera, su complejidad no se justifica. La escribió el autor y no es un *linter* de terceros; la comparación con un LLM juez queda para Capstone II.

4.3 Resultados: eficacia del auditor

Donde: TP, TN, FP, FN: verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos; $\mathrm{Precisión} = TP/(TP+FP)$; $\mathrm{Recall} = TP/(TP+FN)$; $\mathrm{VPN} = TN/(TN+FN)$.

Este 100 % no prueba que el auditor generalice: el banco lo produce un generador escrito por el mismo autor que el auditor (evaluación circular), con patrones que este conoce. Por eso se añadió una **evaluación independiente** con 13 scripts escritos a mano (9 con fuga y 4 limpios), con estilos que el

Tabla 4.1. *Banco de control: 192 scripts con verdad conocida.*

Detector	TP	TN	FP	FN	Precisión	Recall	VPN
AST (estático)	96	96	0	0	1,00	1,00	1,00
Ejecución	96	96	0	0	1,00	1,00	1,00
Híbrido	96	96	0	0	1,00	1,00	1,00

generador no produce: `make_pipeline`, DataFrames, funciones auxiliares, alias de importación, variables `X_te`, y `fit` y `transform` por separado.

Tabla 4.2. *Evaluación independiente ($n = 13$) y línea base externa (regla simple).*

Conjunto	Detector	TP	FP	FN	Precisión	VPN	Recall [IC95 %]
Independiente (13)	Regla simple	8	1	1	0,89	0,75	0,89 [0,52; 1,00]
	AST	8	0	1	1,00	0,80	0,89 [0,52; 1,00]
	Ejecución	7	0	2	1,00	0,67	0,78 [0,40; 0,97]
	Híbrido	9	0	0	1,00	1,00	1,00 [0,66; 1,00]
Banco (192)	Regla simple	72	0	24	1,00	0,80	0,75 [0,65; 0,83]
	Híbrido	96	0	0	1,00	1,00	1,00

Nota. IC95 %: intervalo exacto de Clopper–Pearson para el recall. Conjunto independiente: 9 scripts con fuga y 4 limpios.

La regla simple rinde sorprendentemente bien en los casos manuales (recall 0,89, igual que el AST), pero con un falso positivo (declara fuga en el script que escala *después* de dividir) y deja pasar el alias de importación; en el banco pierde 24 de 96 fugas. El AST falla con el alias (`StandardScaler as SS`); la ejecución falla con ajustes que no usan todas las filas (el escalador re-ajustado con las filas de prueba y el ajustado con el 70 % de entrenamiento antes de la validación cruzada). Fallan en casos distintos, lo que respalda el diseño híbrido. Pero los intervalos se solapan ampliamente: con $n = 9$ casos con fuga, el límite inferior del recall híbrido es 0,66, y la ventaja sobre la línea base es un margen que Capstone II debe confirmar, no una mejora demostrada. La reparación automática resolvió 4 de los 9 scripts con fuga; en los otros 5 declaró «requiere revisión manual» y no tocó el código.

4.4 Resultados preliminares: Sesgo de Optimismo

Nota. *Media de 2 datasets (se excluyó el par degenerado).

Se observan tres hechos. **(1) En clasificación, $\Delta\mathrm{Opt}$ es prácticamente nulo**: en 48 pares va de $-0{,}0027$ a $+0{,}0034$, con un AUC limpio mediano de 0,991 (efecto techo). **(2) En regresión aparece optimismo, concentrado en la selección con `california_housing`**: R^2 de 0,690 (sin fuga) frente a 0,731 (con fuga) en RandomForest, y de 0,615 frente a 0,689 en MLP; en los 47 pares el rango es de $-0{,}023$ a $+0{,}074$. **(3) Escalado y árboles:** en RandomForest y HistGradientBoosting, en los 6 datasets, el máximo $|\Delta\mathrm{Opt}|$ por escalado es 0,0002, como predice la teoría: una transformación afín creciente no cambia la estructura de un árbol.

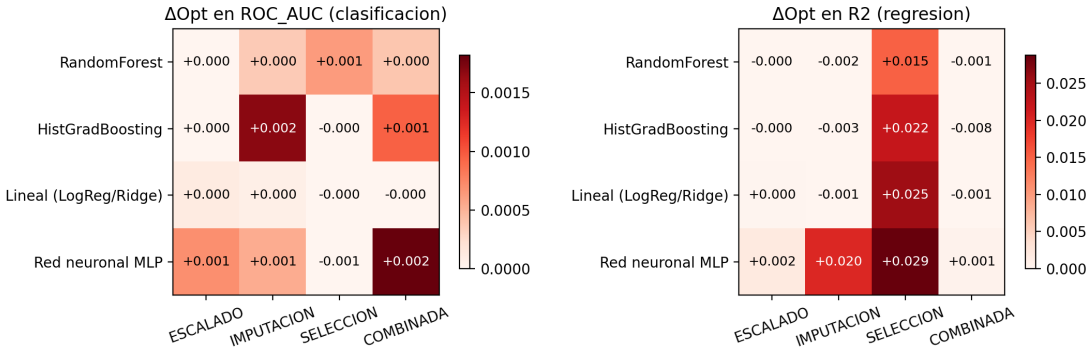


Figura 4.1. $\Delta\mathrm{Opt}$ *medio por familia y tipo de fuga (ROC-AUC en clasificación,* R^2 *en regresión). El par degenerado (D5, MLP, combinada) está excluido.*

Tabla 4.3. $\Delta\mathrm{Opt}$ *medio, promedio sobre los datasets.*

Tarea	Familia	Escalado	Imputación	Selección	Combinada
Clasificación (AUC)	RandomForest	0,0000	0,0004	0,0006	0,0004
	Gradient Boosting	0,0000	0,0017	$-0{,}0001$	0,0010
	Lineal	0,0001	0,0001	$-0{,}0001$	0,0000
	MLP	0,0007	0,0005	$-0{,}0009$	0,0018
Regresión (R^2)	RandomForest	$-0{,}0001$	$-0{,}0024$	0,0150	$-0{,}0014$
	Gradient Boosting	0,0000	$-0{,}0033$	0,0220	$-0{,}0082$
	Lineal	0,0001	$-0{,}0008$	0,0255	$-0{,}0007$
	MLP	0,0019	0,0200	0,0288	0,0006*

Tabla 4.4. *Pruebas por familia en regresión (fugas simples,* $n = 9$ *pares).*

Familia	$\Delta\mathrm{Opt}$ medio	IC95 % bootstrap	d de Cohen	p Wilcoxon	p Holm
RandomForest	0,0042	$[-0{,}0023;$ 0,0142]	0,29	0,320	1,000
Gradient Boosting	0,0062	$[-0{,}0021;$ 0,0199]	0,32	0,371	1,000
Lineal (Ridge)	0,0083	$[-0{,}0015;$ 0,0250]	0,34	0,285	1,000
MLP	0,0169	[0,0027; 0,0346]	0,65	**0,037**	0,297

Nota. Hipótesis alternativa: $\Delta\mathrm{Opt} > 0$ (Wilcoxon unilateral). IC95 %: bootstrap percentil de la media con 10.000 remuestreos (semilla 42). p Holm: ajuste de Holm sobre las 8 pruebas (2 tareas $\times$ 4 familias). Se usan las fugas simples porque la combinada repite esos mecanismos.

En clasificación, el p de Wilcoxon de todas las familias es $\geq 0{,}148$ y todos los intervalos incluyen el cero. En regresión, la MLP es la única familia cuyo intervalo excluye el cero y cuyo p sin corregir es menor que 0,05, pero tras el ajuste de Holm (Holm, 1979) su p sube a 0,297. La prueba de Friedman entre familias da $p = 0{,}84$ en clasificación y $p = 0{,}041$ en regresión (rangos medios, donde 1 indica el mayor optimismo: RandomForest 3,39; Gradient Boosting 2,72; Lineal 2,11; MLP 1,78), que con la corrección de Bonferroni para dos pruebas pasa a 0,082. Por eso las diferencias entre familias se reportan como **exploratorias** (Demšar, 2006).

4.5 Control positivo: cuándo la selección con fuga sí daña

Para comprobar que $\Delta\mathrm{Opt}$ detecta la fuga cuando esta importa, se repitió solo la selección con 0, 50, 200 y 1000 columnas de ruido, en dos escenarios: *real* (etiquetas verdaderas) y *señal nula* (etiquetas permutadas al azar, de modo que la verdad es AUC $= 0{,}5$ y $R^2 \leq 0$).

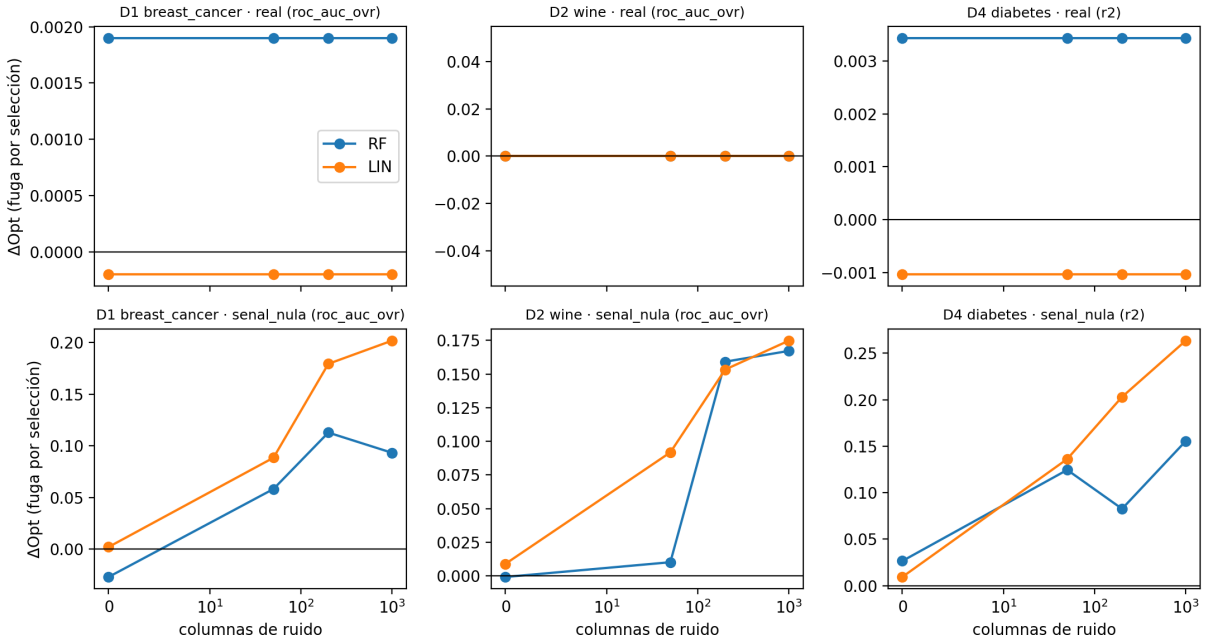


Figura 4.2. $\Delta\mathrm{Opt}$ *de la selección con fuga según el número de columnas de ruido. Fila superior: etiquetas reales. Fila inferior: señal nula (control positivo).*

Con etiquetas reales, $\Delta\mathrm{Opt}$ **no cambia** con el ruido (por ejemplo, `breast_cancer` con RandomForest: 0,0019 en los cuatro niveles), porque con al menos 8 variables muy informativas el selector nunca elige columnas de ruido. Con señal nula crece con el ruido hasta $+0{,}20$ **de AUC** (`breast_cancer`, lineal, 1000 columnas) y $+0{,}26$ **de** R^2 (`diabetes`, lineal): el selector «encuentra» columnas de ruido que, por azar, parecen predecir.

5. INTERPRETACIÓN Y JUICIO CRÍTICO

Qué indican los resultados. (a) El auditor reconoce con fiabilidad los patrones para los que fue diseñado, pero el 100 % del banco **no** es evidencia de generalización: la prueba independiente baja el recall del AST a 0,89 y el del verificador a 0,78, y cada uno falla donde el otro acierta. La conclusión sostenible es que **la combinación es más robusta que cada parte**, no que el auditor sea casi perfecto. (b) La fuga de preprocesamiento y selección tuvo un efecto pequeño en los datasets elegidos, no porque sea inocua sino porque estos datos son «fáciles»: señal fuerte y AUC en el techo. El control positivo muestra que, cuando la señal es débil frente al ruido, el mismo mecanismo infla la métrica hasta 0,20–0,26; la gravedad depende de la relación entre señal y ruido y no es una propiedad fija de cada tipo de fuga ni de cada familia de modelos. (c) De las dos hipótesis de la reformulación, la del escalado en árboles se confirma con claridad; la de que la red neuronal es la más vulnerable tiene respaldo débil: se observa en regresión ($\Delta\mathrm{Opt}$ medio 0,017; $d = 0{,}65$) pero no en clasificación, y no resiste la corrección por multiplicidad.

Implicaciones para la factibilidad. El proyecto es factible como instrumento (detectar, reparar y medir) y se puede defender con lo hecho. Es menos sólido como afirmación empírica sobre «qué modelos sufren más», porque hay solo 3 datasets por tarea y 9 pares por familia. Por eso el objetivo OE3 se reformula para Capstone II: estimar $\Delta\mathrm{Opt}$ **condicionado a la razón señal/ruido**, y no como un número único por familia.

Limitaciones y amenazas a la validez. *Datos:* seis datasets clásicos que los LLM probablemente conocen, un submuestreo de `california_housing` y un efecto techo en clasificación. *Método:* el generador y el auditor tienen el mismo autor (circularidad), la prueba independiente tiene 13 casos, la exclusión de un par degenerado fue posterior a ver el problema y los valores p no están corregidos. *Generalización:* los resultados valen para pipelines sintéticos de scikit-learn; **aún no se sabe qué tan frecuente es la fuga en código escrito por LLM reales**, que es la pregunta motivadora del proyecto, ni cómo se comporta el auditor ante su variedad de estilos.

Riesgos para Capstone II y ajustes propuestos. La Tabla 5.1 los resume junto con la evidencia que los respalda.

Tabla 5.1. *Riesgos observados, evidencia y ajustes para Capstone II.*

Riesgo	Evidencia	Ajuste
El código de LLM reales es más variado que las plantillas y el recall caerá	AST 0,89 y ejecución 0,78 en código escrito a mano	Ampliar la prueba independiente a ≥ 60 scripts y, con los LLM reales, etiquetar una muestra a mano con revisión cruzada
Alias, funciones auxiliares y reparaciones manuales (solo 4/9 reparados)	Un caso con alias; cuatro casos sin reparación automática	Resolver alias de importación; ampliar el refactorizador a `ColumnTransformer` y funciones
Efecto techo y pocos datasets	AUC mediano 0,991; $\lvert\Delta\mathrm{Opt}\rvert \leq 0{,}0034$ en clasificación	Incorporar datasets de señal débil o con muchas más variables que filas; informar $\Delta\mathrm{Opt}$ por tramo de señal/ruido
Ejecutar código de LLM es riesgoso	—	Ejecutar solo en contenedor sin red ni credenciales; la plataforma web seguirá sin ejecutar código
Costo y términos de servicio de las APIs	Script de generación sin probar	Medir tokens con 2 modelos antes de escalar; revisar términos de publicación de salidas

6. APLICABILIDAD Y USO RESPONSABLE

6.1 Contexto de uso

Los usuarios previstos son (1) analistas y estudiantes de ciencia de datos que reciben pipelines de un asistente de IA y (2) equipos que quieren una revisión automática antes de integrar código a un repositorio. La decisión que informa el auditor es: *aceptar, corregir o derivar a revisión humana* un pipeline. Los tres productos comparten el mismo núcleo (`03_auditor/`), de modo que un script recibe el mismo veredicto por cualquier vía.

6.2 Propuesta de implementación

Skills para agentes (`05_skills/`). `audit-ml-leakage` audita y repara scripts existentes; `create-leakage-free-pipeline` impone reglas al crear código (todo preprocesamiento dentro de un `Pipeline`, prueba final apartada, semillas fijas) y exige autoverificarse con el auditor antes de entregar. Cada Skill es una carpeta autocontenida que se copia al directorio de Skills del agente. La herramienta de línea de comandos devuelve el código de salida 0 (limpio) o 1 (fuga), por lo que también puede bloquear una integración continua.

Plataforma web en Streamlit (`06_web_app/`), pensada para quien no usa agentes de programación. Su diseño separa tres capas: la interfaz (`app.py`), que solo dibuja pantallas; la lógica (`app_logic.py`), que valida la entrada, invoca al auditor, resalta líneas y calcula diferencias, y que se prueba sin navegador; y el núcleo auditor compartido. El flujo de uso tiene cuatro pestañas: (1) *auditar y reparar*: el usuario pega o sube un `.py`, ve en rojo las líneas con fuga y la regla que las explica, y puede descargar la versión corregida junto con sus diferencias; (2) *cuánto infla la fuga*: consulta $\Delta\mathrm{Opt}$ para cualquier combinación de dataset, familia y tipo de fuga del experimento; (3) *resultados del experimento*: eficacia del auditor y pruebas por familia; y (4) *guía*: qué es la fuga, ejemplos y limitaciones.

Requisitos: Python (probado con la versión 3.13), scikit-learn y Streamlit, con versiones fijas en `requirements.txt`. *Condiciones de uso:* scripts tabulares con scikit-learn; los notebooks se exportan a `.py`. *Verificación hecha:* pruebas automáticas que cubren el auditor, la reparación, las Skills y el arranque de la plataforma web. *Monitoreo:* registrar los falsos negativos que reporten los usuarios y convertirlos en casos de prueba. **Aún no se ha hecho una prueba de uso con personas**; el producto es un prototipo y esa prueba queda para Capstone II.

6.3 Riesgos éticos y medidas

La Tabla 6.1 identifica los riesgos propios de este caso y la medida concreta que los aborda, junto con la forma de comprobar que la medida existe.

Nota. No se evalúa la equidad de los modelos entrenados porque no es el objeto del estudio; se declara como límite.

Tabla 6.1. *Riesgos de uso responsable, medidas y verificación.*

Riesgo	Por qué aplica a este proyecto	Medida concreta	Verificación
Falsa seguridad	«0 fugas» puede leerse como «sin fuga», pero no se ven fugas temporales, de dominio, por duplicados ni código dinámico	La web, la línea de comandos y las Skills muestran que el resultado significa «sin evidencia de las reglas R-Split, R-CV y R-Test»	Texto visible en la pestaña 1 y en ambos `SKILL.md`
Ejecución de código no confiable	El código de LLM o de usuarios puede ser malicioso	La web solo lee el código (AST) y nunca lo ejecuta; rechaza entradas de más de 200.000 caracteres y escapa el HTML; el código de LLM se ejecutará solo en un contenedor sin red ni credenciales	Pruebas automáticas de escape de HTML y de límite de tamaño
Datos sensibles	`breast_cancer` y `diabetes` son datos clínicos	Se usan solo conjuntos públicos y anonimizados de scikit-learn, sin fines clínicos; la web no guarda el código que se le entrega	Carga desde `sklearn.datasets`; sin almacenamiento en la aplicación
Sesgo del instrumento	El auditor se probó con estilos de un solo autor	Evaluación independiente separada del banco; se informan los fallos conocidos (alias de importación)	Tabla 4.2; casos 05 y 09
Términos de servicio	Las salidas de las APIs de LLM pueden tener restricciones de publicación	Revisar los términos de cada proveedor antes de publicar el Conjunto A	Registro en `decision_log.md`

Declaración de uso de inteligencia artificial. El problema, las ideas, el diseño del estudio y el borrador del informe son del autor. Se utilizó un asistente de programación basado en IA como apoyo para implementar y comentar el código, organizar el repositorio y editar la redacción a partir de ese borrador. Las cifras provienen de la ejecución del código y se contrastaron con los archivos de resultados; el autor es responsable del contenido y de su defensa.

7. CONCLUSIONES

La pregunta de investigación tiene, con la evidencia de Capstone I, una respuesta afirmativa pero acotada. **Detectar** es posible: el auditor determinista reconoce la fuga de preprocesamiento y selección en scripts de scikit-learn, y en una prueba independiente pequeña el diseño híbrido detectó los 9 casos con fuga, aunque ninguno de sus dos componentes lo logró por sí solo (OE2). **Reparar sin cambiar el modelo** es posible en los patrones habituales, pero no en general: 96 de 96 en el banco y 4 de 9 en código escrito a mano (OE4). **Cuantificar** es posible con $\Delta\mathrm{Opt}$, un indicador pareado que separa la fuga del sobreajuste; su magnitud no es una propiedad fija de cada familia de modelos, sino que depende de la relación entre señal y ruido: casi nula con datos de señal fuerte y de hasta 0,20 de AUC con señal nula (OE3). Los productos, dos Skills y una plataforma web, están operativos como prototipo (OE4).

Lo que falta, y es lo que da sentido al proyecto, es medir la fuga en código escrito por LLM reales. Para Capstone II se fijan de antemano los umbrales, se amplía la evaluación independiente y se reformula OE3 en términos de señal y ruido. Todas las cifras de este informe y el código que las produce están en el repositorio y se regeneran con un solo comando.

REFERENCIAS

AlOmar, E. A., DeMario, C., Shagawat, R., & Kreiser, B. (2025). LeakageDetector: An open source data leakage analysis tool in machine learning pipelines. *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 844–849. https://doi.org/10.1109/SANER64311.2025.00089

Ambroise, C., & McLachlan, G. J. (2002). Selection bias in gene extraction on the basis of microarray gene-expression data. *Proceedings of the National Academy of Sciences*, *99*(10), 6562–6566. https://doi.org/10.1073/pnas.102102699

Cawley, G. C., & Talbot, N. L. C. (2010). On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, *11*, 2079–2107. https://jmlr.org/papers/v11/cawley10a.html

Chan, J. S., Chowdhury, N., Jaffe, O., Aung, J., Sherburn, D., Mays, E., Starace, G., Liu, K., Maksin, L., Patwardhan, T., Weng, L., & Mądry, A. (2025). MLE-bench: Evaluating machine learning agents on machine learning engineering. *The Thirteenth International Conference on Learning Representations (ICLR 2025)*. https://openreview.net/forum?id=6s5uXNWGIh

Clopper, C. J., & Pearson, E. S. (1934). The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, *26*(4), 404–413. https://doi.org/10.1093/biomet/26.4.404

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, *7*, 1–30. https://jmlr.org/papers/v7/demsar06a.html

Draoui, T., Mkaouer, M. W., & Newman, C. (2026). Improving data leakage detection in machine learning notebooks through static slicing and structured LLM prompts. *Proceedings of the ACM on Software Engineering*, *3*(FSE), 4371–4392. https://doi.org/10.1145/3808199

Drobnjaković, F., Subotić, P., & Urban, C. (2024). An abstract interpretation-based data leakage static analysis. *Theoretical Aspects of Software Engineering (TASE 2024)*, *14777*, 109–126. https://doi.org/10.1007/978-3-031-64626-3_7

Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2.ª ed.). Springer. https://doi.org/10.1007/978-0-387-84858-7

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, *28*(1), 75–105. https://doi.org/10.2307/25148625

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, *6*(2), 65–70.

Hong, S., Lin, Y., Liu, B., Liu, B., Wu, B., Zhang, C., Li, D., Chen, J., Zhang, J., Wang, J., Zhang, L., Zhang, L., Yang, M., Zhuge, M., Guo, T., Zhou, T., Tao, W., Tang, R., et al. (2025). Data Interpreter: An LLM agent for data science. *Findings of the Association for Computational Linguistics: ACL 2025*, 19796–19821. https://doi.org/10.18653/v1/2025.findings-acl.1016

Huang, Q., Vora, J., Liang, P., & Leskovec, J. (2024). MLAgentBench: Evaluating language agents on machine learning experimentation. *Proceedings of the 41st International Conference on Machine Learning (PMLR 235)*, 20271–20309. https://proceedings.mlr.press/v235/huang24y.html

Kapoor, S., Cantrell, E. M., Peng, K., Pham, T. H., Bail, C. A., Gundersen, O. E., Hofman, J. M., Hullman, J., Lones, M. A., Malik, M. M., Nanayakkara, P., Poldrack, R. A., Raji, I. D., Roberts, M., Salganik, M. J., Serra-Garcia, M., Stewart, B. M., Vandewiele, G., & Narayanan, A. (2024). REFORMS: Consensus-based recommendations for machine-learning-based science. *Science Advances*, *10*(18), Artículo eadk3452. https://doi.org/10.1126/sciadv.adk3452

Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, *4*(9), Artículo 100804. https://doi.org/10.1016/j.patter.2023.100804

Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, *6*(4), Artículo 15. https://doi.org/10.1145/2382577.2382579

Kumar, S., Lones, M. A., Maarek, M., & Zantout, H. (2025). *Navigating pitfalls: Evaluating LLMs in machine learning programming education*. arXiv. https://doi.org/10.48550/arXiv.2505.18220

Lai, Y., Li, C., Wang, Y., Zhang, T., Zhong, R., Zettlemoyer, L., Yih, W.-t., Fried, D., Wang, S., & Yu, T. (2023). DS-1000: A natural and reliable benchmark for data science code generation. *Proceedings of the 40th International Conference on Machine Learning (PMLR 202)*, 18319–18345. https://proceedings.mlr.press/v202/lai23b.html

Lones, M. A. (2024). Avoiding common machine learning pitfalls. *Patterns*, *5*(10), Artículo 101046. https://doi.org/10.1016/j.patter.2024.101046

Nam, J., Yoon, J., Chen, J., Shin, J., Arık, S. Ö., & Pfister, T. (2025). MLE-STAR: Machine learning engineering agent via search and targeted refinement. *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 129290–129310. https://doi.org/10.52202/085713-3889

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Varma, S., & Simon, R. (2006). Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, *7*, 91. https://doi.org/10.1186/1471-2105-7-91

Yang, C., Brower-Sinning, R. A., Lewis, G., & Kästner, C. (2022). Data leakage in notebooks: Static detection and better processes. *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE 2022)*, 1–12. https://doi.org/10.1145/3551349.3556918

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 46595–46623. https://doi.org/10.52202/075280-2020

ANEXOS

ANEXO A. REPRODUCIBILIDAD Y REPOSITORIO

Repositorio: https://github.com/witman92/ml-leakage-auditor-capstone.

Todas las tablas y figuras de este informe se generan en `04_experiments/results/` y se reproducen con:

```
pip install -r requirements.txt
python 04_experiments/leakage_demo.py   # ejemplo visual (Figura 1.2)
python run_all.py                       # fases 0-5, estadistica, figuras y pruebas
python run_all.py --rapido              # verificacion rapida (2 datasets x 2 familias)
streamlit run 06_web_app/app.py         # plataforma web
```

Estructura del repositorio. Cada carpeta numerada corresponde a una fase del protocolo (Tabla 3.1):

```
ml-leakage-auditor-capstone/
|-- run_all.py                  # reproduce todo con un comando
|-- requirements.txt            # versiones fijas (Python 3.13)
|-- 01_data/                    # Fase 0: carga de los 6 datasets de scikit-learn
|-- 02_pipelines/               # Fase 1: banco de 96 pares (controlled_pairs/),
|                               #   reparados (repaired_pairs/) y Conjunto A (llm_generated/)
|-- 03_auditor/                 # Fase 2: AST, verificador en ejecucion, refactorizador, CLI
|-- 04_experiments/             # Fases 3-4: benchmark, EDA, estadistica, demo, linea base
|   `-- results/                #   tablas (.csv), figuras (.png), manifiesto de ejecucion
|-- 05_skills/                  # Fase 5: dos Agent Skills autocontenidas
|-- 06_web_app/                 # Fase 5: plataforma web en Streamlit
|-- 07_report/                  # informe LaTeX, registro de decisiones, checklist
`-- 08_tests/                   # pruebas automaticas y 13 casos escritos a mano
```

Cada script está organizado en bloques numerados (`#%% PASO n`) que se pueden ejecutar uno a uno en VS Code o en Colab, y cada bloque explica de dónde vienen los datos, qué hace la función, qué métrica o gráfico produce y cómo interpretarlo. `run_benchmark.py` escribe además `run_manifest.json` con las versiones de las librerías y la semilla, y ordena todas las tablas de salida, de modo que dos ejecuciones producen archivos idénticos. La carpeta se abre en VS Code con `capstone.code-workspace`, que trae una tarea por fase. Las decisiones y desviaciones están en `07_report/decision_log.md`.